



# Networking

## Lecture 7



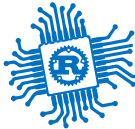
# Networking

- OSI Network Stack
- Wi-Fi
- TCP/IP
- Raspberry Pi W
- Protocols



# OSI Network Stack

Open Standard for Intercommunication



# Bibliography

for this section

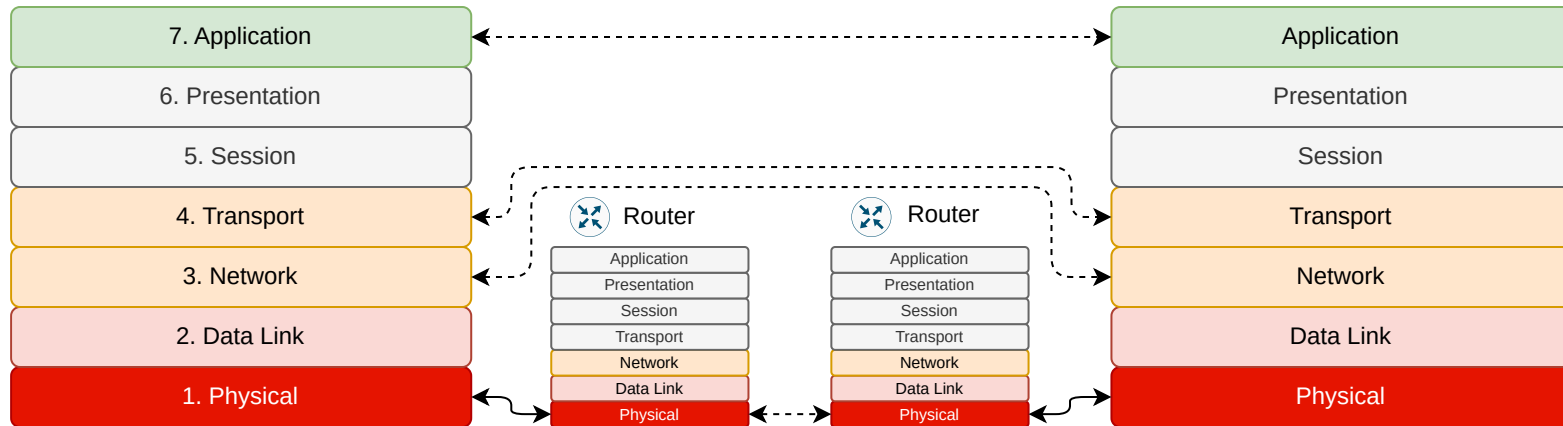
**Andrew Tanenbaum**, *Computer Networks (5th edition)*

- Chapter 1 - *Introduction*
  - Subchapter 1.1 - *Uses of Computer Networks*
  - Subchapter 1.2 - *Network Hardware*
  - Subchapter 1.3 - *Network Software*
  - Subchapter 1.4.1 - *The OSI Reference Model*



# Standardized Interfaces

7 layers, each one communicates with its counterpart

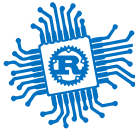


- **L1** *hardware*, sends and receives data on the physical media
- **L2** *hardware* and *driver* sends and receives data from a device that it is directly connected to
- **L3** *driver* sends and receives data from devices not directly connected to using L2 from device to device
- **L4** *driver* connects the applications to the networking stack
- **L5/L6** not used
- **L7** is the *application*



# Wi-Fi

Wireless Network

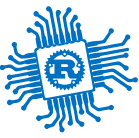


# Bibliography

for this section

**Andrew Tanenbaum**, *Computer Networks (5th edition)*

- Chapter 1 - *Introduction*
  - Subchapter 1.5.3 - *Wireless LANs: 802.11*



# Wi-Fi

- Wireless Network
- *L2* (Data Link) Protocol
- Devices
  - **AP** - Access Point
    - acts as a hub or switch
    - handles authentication
  - **Device** - The device that connects to the network
- Frequencies
  - 2.4 GHz
  - 5 GHz







# Wireless Network Connection

## security

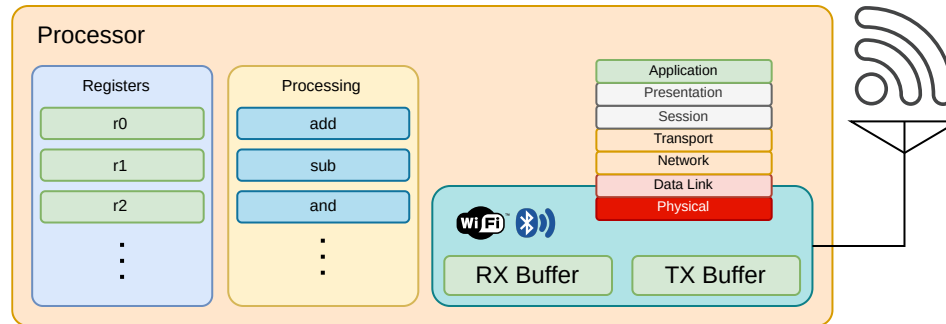
- **Open** - everyone receives all the communication
- **WEP** - all data is encrypted with the same key, everyone who knows the keys can read the data
- **WPA 1/2/3 (Personal)** - each **device** has a different encryption key shared with the **AP**
  - the **device** authenticates with the **AP** by using the network *passkey*
  - the **device** and the **AP** exchange a symmetric encryption key
- **WPA 1/2/3 Enterprise** - each **device** has a different encryption key shared with the **AP**
  - the **AP** provides a certificate to the **device** proving its authenticity
  - the **device** authenticates using username and password or a private key
  - the **device** and the **AP** exchange a symmetric encryption key



# Integrated Network Device

the network device is integrated into the MCU

- a radio peripheral
  - knows how to emit and receive in 2.4 and 5 GHz
  - is controlled by software
  - can generate signals for Wi-Fi, BLE, 802.15.4, 6LoPAN, Thread
- it knows how to transmit and receive buffers (*L1*)
- some devices know *L2*

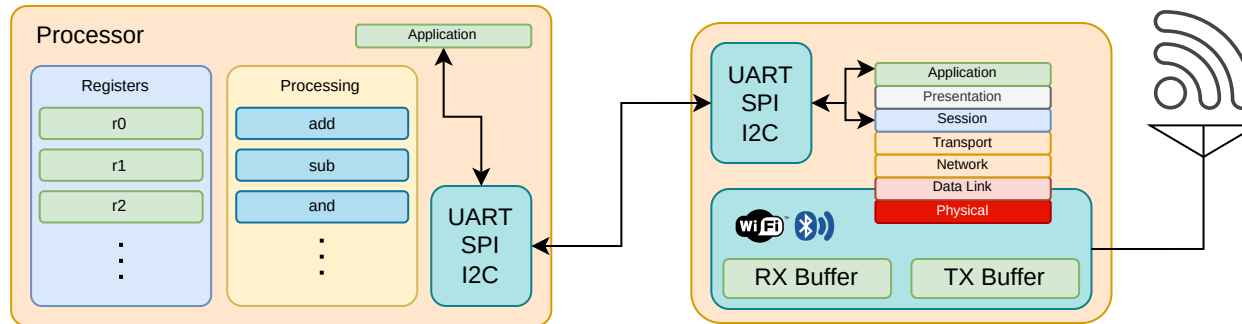




# Discrete Network Device

the network device is connected into the MCU

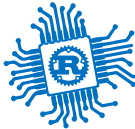
- the MCU is connected to an external Wi-Fi/BLE device
- transport over UART, SPI or I2C
- most devices knows
  - *L3* - provides *socket*
  - *L4* - provides TCP/UDP *sockets*
  - *L7* - provides application functions (usually *HTTP* and *MQTT*)





# TCP/IP Stack

Transport Control Protocol over Internet Protocol

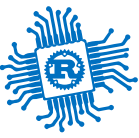


# Bibliography

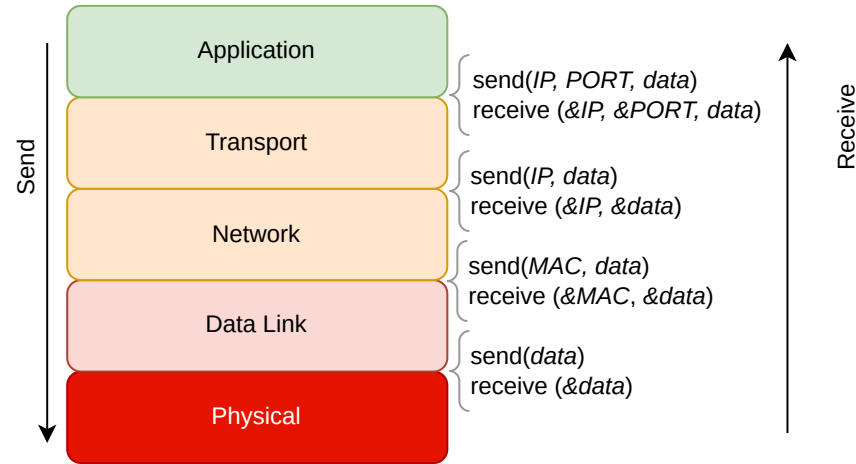
for this section

**Andrew Tanenbaum**, *Computer networks (5th edition)*

- Chapter 1 - *Introduction*
  - Subchapter 1.4.2 - *The TCP/IP Reference Model*



# TCP/IP Stack



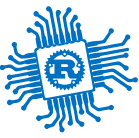
\*the initial TCP/IP stack did not make any difference between the *Physical* and the *Data Link* layers



# Data Link Layer

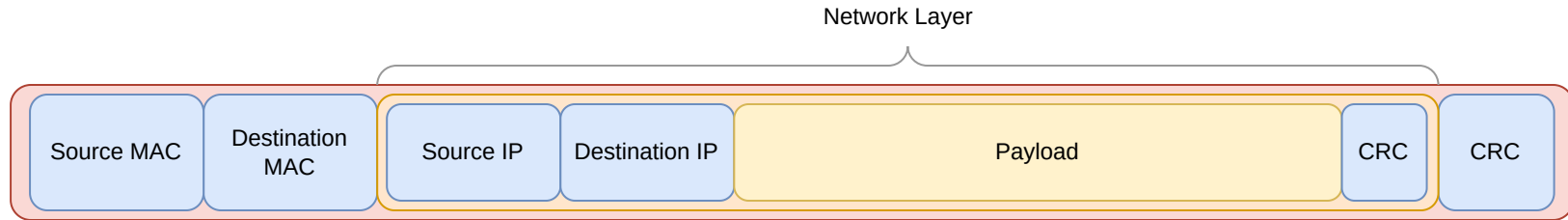
- very similar for Ethernet and Wi-Fi (HDLC)
- uses *Media Access Control (MAC)* addresses
- sends and receives *frames* from other devices directly connected to the same network





# Network Layer

- Internet Protocol
  - uses *Internet Protocol* (IP) addresses
    - *IPv4* - 32 bits
    - *IPv6* - 128 bits
- sends and receives *packets* from other devices remotely

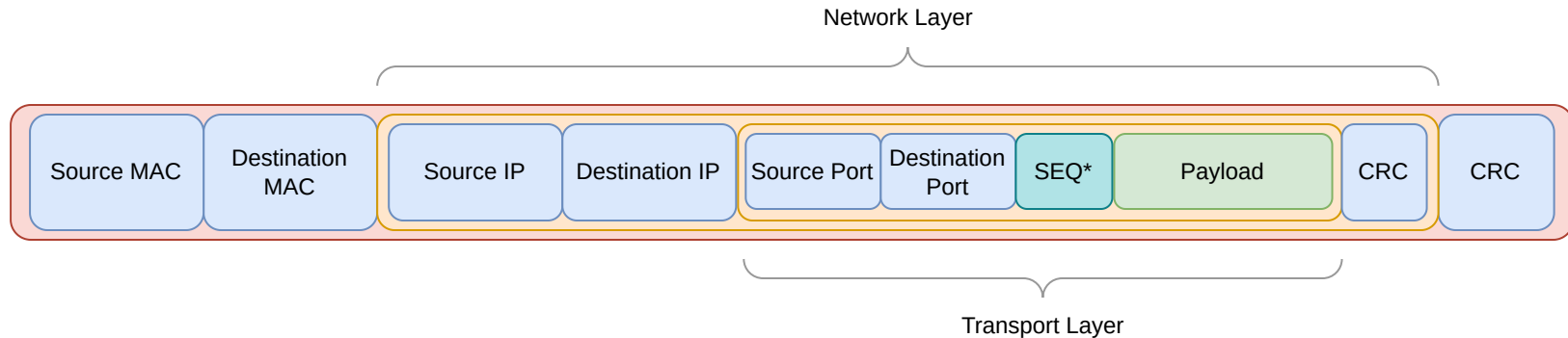






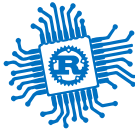
# Transport Layer

- Two protocols
  - *Transport Control Protocol (TCP)* - stream of data, makes sure it gets to the destination
  - *User Datagram Protocol (UDP)* - *fire and forget*, best effort do deliver the packet
- uses *Ports* to identify the destination and source application
- sends and receives *packets*





# Raspberry Pi Pico W



# Bibliography

for this section

**Andrew Tanenbaum**, *Computer networks (5th edition)*

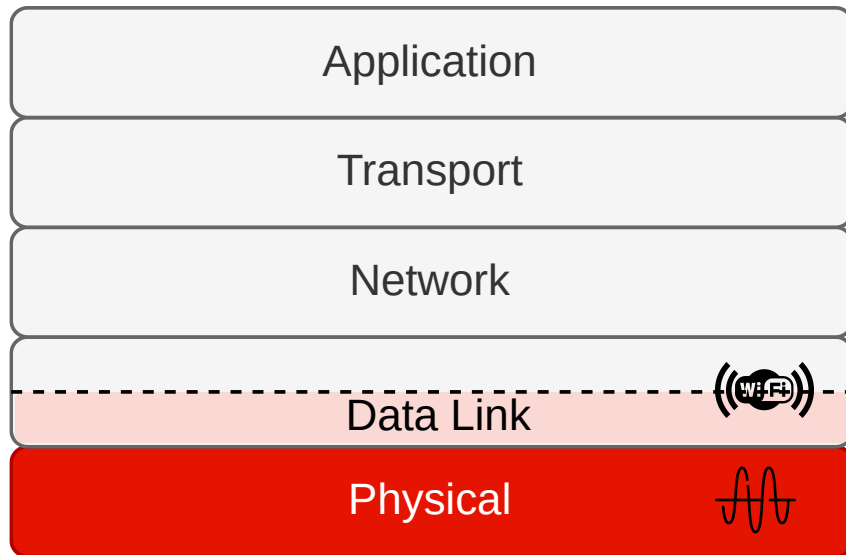
- Chapter 7 - *Application Layer*
  - Subchapter 7.1 - *DNS - Domain Name System*



# Raspberry Pi Pico W

uses a discrete Wi-Fi chip

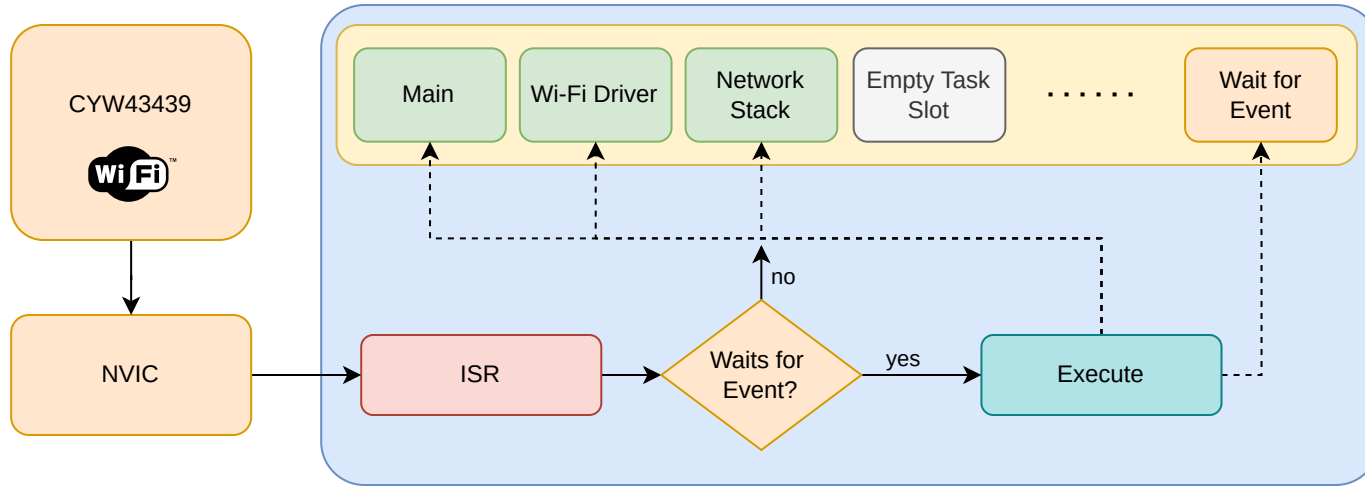
- Wi-Fi and BLE provided by CYW43439 made by *Infineon*
- connected over SPI/PI0
- Wi-Fi 4 (802.11n), 2.4 GHz
  - WPA 3
  - SoftAP (4 clients)
  - Device
- BLE 5.2
  - Central
  - Peripheral
  - Bluetooth Classic
- Provides *L2* - allows sending of *Ethernet* (MAC) frames





# Tasks

tasks that run when using Wi-Fi





1. Load the *firmware* into the `.data` section.

## 2. Use PIO0 as SPI device

The diagram illustrates the memory layout of the RP2040 Boot Loader, organized into two main sections: the Interrupt Vector Table and Code and Data.

**Interrupt Vector Table:** This section contains the following entries:

- 0x000: RP2040 Boot Loader *.boot\_loader*
- 0x100: Initial Stack Address
- 0x104: Reset Handler
- 0x108: NMI Handler
- 0x10c: HardFault Handler
- 0x12c: SVC Handler
- 0x138: PendSV
- 0x13c: SysTick Handler
- 0x140: ISR 0
- 0x144: ISR 1
- .....
- 0x1bc: ISR 31
- 0x1c0: (Empty entry)

**Code and Data:** This section contains the following components:

- Code .text:** The main code area, which is the target of jumps from the HardFault Handler, SVC Handler, and ISR 1.
- Data .rodata & .data:** The data area.
- CYW43439 Firmware:** The final component in the memory layout.

**Jump Targets:**

- Jump to main:** Indicated by a dashed line from the Reset Handler (0x104) to the Initial Stack Address (0x100).
- Jump when IRQ 1 fires:** Indicated by a dashed line from ISR 1 (0x144) to the Code .text area.
- Jump to panic handler:** Indicated by a dashed line from the HardFault Handler (0x10c) to the Code .text area.

\* drawing is not at scale, code and data are significantly greater than the interrupt vector



# CYW43439 API

the embassy driver

## 3. Write a task for the Wi-Fi driver

```
#[embassy_executor::task]
async fn wifi_task(runner: cyw43::Runner<'static, Output<'static>, PioSpi<'static, PIO0, 0, DMA_CH0>>>) -> ! {
    runner.run().await
}
```

## 4. Start the driver

```
1 static STATE: StaticCell<cyw43::State> = StaticCell::new();
2 let state = STATE.init(cyw43::State::new());
3 let (_net_device, mut control, runner) = cyw43::new(state, pwr, spi, fw).await;
4 unwrap!(spawner.spawn(wifi_task(runner)));
```

## 5. Init the device

```
1 control.init(clm).await;
2 control
3     .set_power_management(PowerManagementMode::PowerSave)
4     .await;
```

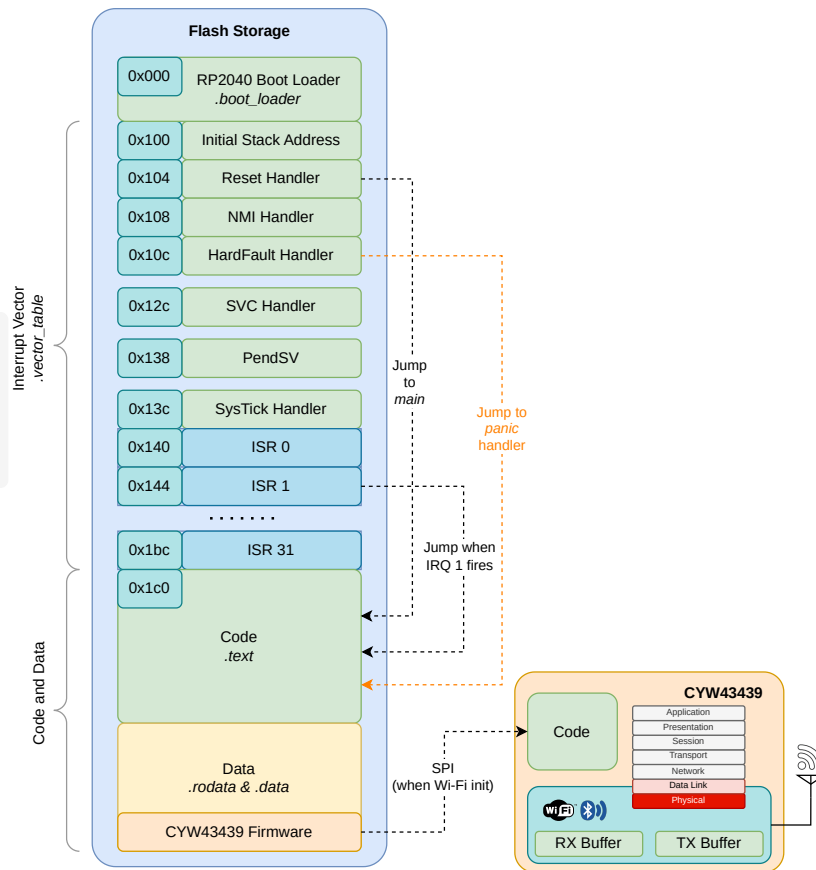


# Flash the firmware

write the firmware to the wifi device

The first action of the *wifi* task is to write the firmware from *.data* to the CY43439 chip.

```
#[embassy_executor::task]
async fn wifi_task(/* ... */) -> ! {
    runner.run().await
}
```







# Wi-Fi AP Mode

Start an **AP** and allow other devices to connect.

*Open Network* (not a very good idea)

- network SSID
- channel number

```
control.start_ap_open("Network SSID", 5).await;
```

*WPA network*

- network SSID
- WPA password
- channel number

```
control.start_ap_wpa2("Network SSID", "WPA password", 5).await;
```



# Wi-Fi Device Mode

Start an **device** and connect to a Wi-Fi network

*Open Network* (not a very good idea)

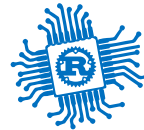
- network SSID

```
control.join_open("network SSID").await;
```

*WPA network*

- network SSID
- network password

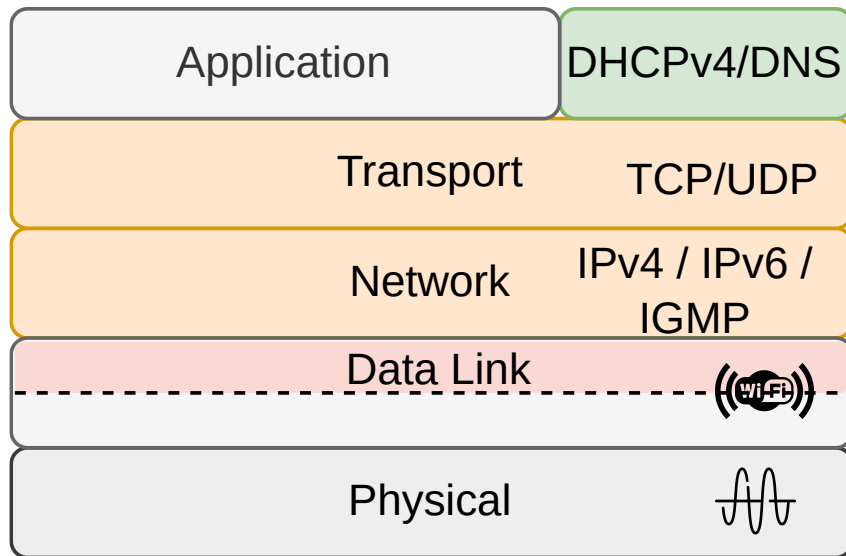
```
match control.join_wpa2("network ssid", "network password").await {  
  Ok(_) => break,  
  Err(err) => {  
    info!("join failed with status={}", err.status);  
  }  
}
```



# Embassy Net

*a smol TCP/IP stack*

- uses smoltcp, embedded (no\_std) TCP/IP stack written in Rust
- *L3*: IPv4, IPv6, IGMPv4 (ping), 6LoWPAN
- *L4*: TCP and UDP
- *L7*: DHCPv4 and DNS





# Embassy Net API

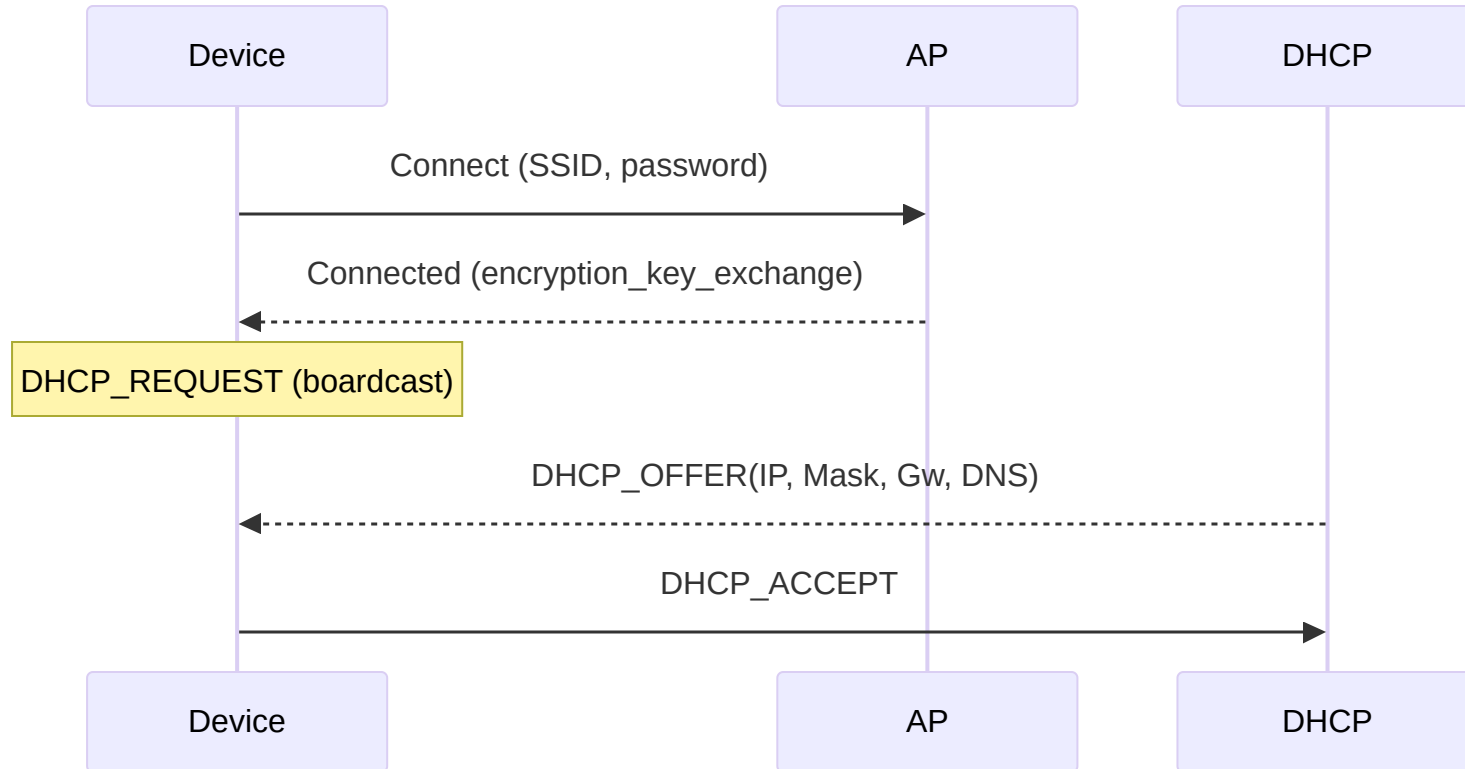
over smoltcp

1. Set how to obtain an IP address
  - self assigned
  - DHCP
2. Start the network stack
3. Use sockets to communicate



# DHCP

Dynamic Host Control Protocol





# Obtain a network address

self assigned or obtain one from a DHCP server

## *Self assigned*

```
1  let config = embassy_net::Config::ipv4_static(embassy_net::StaticConfigV4 {  
2      address: Ipv4Cidr::new(Ipv4Address::new(192, 168, 69, 2), 24),  
3      dns_servers: vec![Ipv4Address::new(8, 8, 8, 8), Ipv4Address::new(1, 1, 1, 1)],  
4      gateway: Some(Ipv4Address::new(192, 168, 69, 1)),  
5  });
```

## *Dynamic Host Control Protocol (DHCP)*

```
1  let config = Config::dhcpv4(Default::default());  
2  
3  // start the network stack  
4  
5  // Wait for DHCP  
6  info!("waiting for DHCP...");  
7  while !stack.is_config_up() {  
8      Timer::after_millis(100).await;  
9  }
```



# Start the network stack

## 1. Write a network task

```
#[embassy_executor::task]
async fn net_task(stack: &'static Stack<cyw43::NetDriver<'static>>) -> ! {
    stack.run().await
}
```

## 2. Start the network stack

```
1 // chosen by fair dice roll. guaranteed to be random.
2 let seed = 0x0123_4567_89ab_cdef;
3
4 // Init network stack
5 static STACK: StaticCell<Stack<cyw43::NetDriver<'static>>> = StaticCell::new();
6 static RESOURCES: StaticCell<StackResources<2>> = StaticCell::new();
7 let stack = &*STACK.init(Stack::new(
8     net_device,
9     config,
10     RESOURCES.init(StackResources::<2>::new()),
11     seed,
12 ));
13
14 unwrap!(spawner.spawn(net_task(stack)));
```



# Query an IP address using DNS

IP address for a domain

- sockets use IP addresses
- to talk to a server, the IP of the server has to be obtained

```
1  let dns = DnsSocket::new(stack);
2  match dns.get_host_by_name("www.example.com", AddrType::IPv4) {
3      Ok(ip) => info!("Ip is {:?}", ip),
4      Err(e) => warn!("failed to retrieve address {:?}", e)
5  }
```





# TCP Server Socket

listening for one single connection

*smoltcp can only listen and accept one client*

```
1  let mut rx_buffer = [0; 4096];
2  let mut tx_buffer = [0; 4096];
3
4  loop {
5      let mut socket = TcpSocket::new(stack, &mut rx_buffer, &mut tx_buffer);
6      socket.set_timeout(Some(Duration::from_secs(10)));
7
8      info!("Listening on TCP:1234...");
9      if let Err(e) = socket.accept(1234).await {
10         warn!("accept error: {:?}", e);
11         continue;
12     }
13
14     info!("Received connection from {:?}", socket.remote_endpoint());
15
16     // handle the connection
17 }
```



# TCP Client Socket

connecting to a server

```
1  let mut rx_buffer = [0; 4096];
2  let mut tx_buffer = [0; 4096];
3
4  loop {
5      let mut socket = TcpSocket::new(stack, &mut rx_buffer, &mut tx_buffer);
6      socket.set_timeout(Some(Duration::from_secs(10)));
7
8      info!("Connecting to TCP 1.2.3.5:1234...");
9      if let Err(e) = socket.connect(IpEndpoint::new(IpAddress::v4(1,2,3,5), 1234)).await {
10         warn!("accept error: {:?}", e);
11         continue;
12     }
13
14     info!("Connected to {:?}", socket.remote_endpoint());
15
16     // handle the connection
17 }
```



# Read from a TCP Socket

read bytes

```
1  let mut buf = [0u8; 4096];
2
3  let n = match socket.read(&mut buf).await {
4      Ok(0) => {
5          warn!("read EOF");
6          break;
7      }
8      Ok(n) => n,
9      Err(e) => {
10         warn!("read error: {:?}", e);
11         break;
12     }
13 };
14
15 // display bytes as a UTF-8 string
16 info!("rxd {}", from_utf8(&buf[..n]).unwrap());
```



# Write to a TCP Socket

write bytes

```
1  let buf = [0u8; 4096];
2
3  // write n bytes to buf
4
5  match socket.write_all(&buf[..n]).await {
6      Ok(()) => {
7          /* write was successful */
8      }
9      Err(e) => {
10         warn!("write error: {:?}", e);
11         break;
12     }
13 };
```



# Listen for UDP Packets

```
1  let mut rx_buffer = [0; 4096];
2  let mut rx_metadata_buffer = [PacketMetadata::EMPTY; 3];
3  let mut tx_buffer = [0; 4096];
4  let mut tx_metadata_buffer = [PacketMetadata::EMPTY; 3];
5
6  let mut buf = [0u8; 4096];
7
8  let mut socket = UdpSocket::new(
9      stack, &mut rx_buffer, &mut rx_metadata_buffer, &mut tx_buffer, &mut tx_metadata_buffer
10 );
11
12 // listen for UDP packet on port 1234
13 if let Err(e) = socket.bind(1234) {
14     warn!("bind error {:?}", e);
15     break;
16 }
17
18 loop {
19     match socket.recv_from(&mut buf) {
20         Ok((n, endpoint)) => {
21             info!("Received from {:?}: {:?}", endpoint, buf[..n]);
22         }
23     }
24 }
```



# Send UDP Packets

```
1  let mut rx_buffer = [0; 4096];
2  let mut rx_metadata_buffer = [PacketMetadata::EMPTY; 3];
3  let mut tx_buffer = [0; 4096];
4  let mut tx_metadata_buffer = [PacketMetadata::EMPTY; 3];
5
6  let mut buf = [0u8; 4096];
7
8  let mut socket = UdpSocket::new(
9      stack, &mut rx_buffer, &mut rx_metadata_buffer, &mut tx_buffer, &mut tx_metadata_buffer
10 );
11
12 info!("Sending to UDP 1.2.3.5:1234...");
13 match socket.send_to(&buf, IpEndpoint::new(IpAddress::v4(1,2,3,5), 1234)) {
14     Ok(()) => {
15         /* send successful */
16     }
17     Err(e) => {
18         warn!("send error: {:?}", e);
19     }
20 }
```



# Protocols

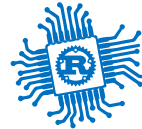


# Libraries

that provide protocols

- MQTT - *MQ Telemetry Transport*
  - publish/subscribe
  - minimq
- CoAP - *Constrained Application Protocol*
  - simplified binary HTTP
  - coap\_lite





# Conclusion

we talked about

- OSI Network Stack
- Wi-Fi
- TCP/IP
- Raspberry Pi W
- Protocols